Migration de données médicales vers MongoDB

Projet Data Engineering – Docker & Cloud

Sommaire

I – Introduction : Contexte de la mission
Contexte du client, problématique liée à la scalabilité des données médicales et objectifs de la mission.

II – Étape 1 : Migration des données vers MongoDB
Présentation du jeu de données médicales, justification du choix de MongoDB, modélisation orientée documents et automatisation du processus de migration.

III – Étape 2 : Conteneurisation de l'application avec Docker
Mise en place de l'architecture Docker, orchestration des services, automatisation du pipeline de migration et gestion du stockage persistant.

IV – Sécurité, authentification et bonnes pratiques
Implémentation des rôles utilisateurs MongoDB, gestion des accès, séparation des privilèges et validation de l'intégrité des données après migration.

V – Étude des solutions de déploiement sur AWS
Analyse des limites d'une solution locale, présentation des services AWS adaptés à MongoDB et comparaison des différentes options de déploiement cloud.

VI – Recommandation d'architecture et services essentiels
Recommandation de déploiement basée sur Amazon DocumentDB et présentation des services fondamentaux garantis par AWS, indépendamment de l'architecture retenue.

VII – Conclusion et perspectives
Synthèse du projet réalisé et perspectives de mise en production et d'évolution de la solution.

I – Introduction : Contexte de la mission

Contexte

Le client dispose d'un ensemble de données médicales de patients stockées sous forme de fichiers tabulaires et gérées dans un environnement local. Avec l'augmentation du volume de données et des usages, cette solution commence à montrer ses limites en termes de performance, de flexibilité et d'évolutivité.

Problématique

La gestion quotidienne des données médicales devient de plus en plus complexe en raison d'un manque de scalabilité, de requêtes coûteuses et d'une difficulté à faire évoluer le schéma de données. L'architecture existante n'est plus adaptée à une montée en charge ni à une exploitation moderne des données.

Objectifs

L'objectif de la mission est de migrer les données médicales vers une base de données MongoDB afin d'obtenir une solution plus flexible et scalable. Cette migration doit être automatisée et conteneurisée à l'aide de Docker, tout en intégrant une réflexion sur les options de déploiement cloud sur AWS.

II – Migration des données vers MongoDB

Présentation du jeu de données

Le jeu de données source est un fichier CSV contenant des informations médicales relatives aux patients et à leurs hospitalisations. Chaque ligne représente une admission, ce qui entraîne la présence de plusieurs enregistrements pour un même patient. Les données sont hétérogènes et susceptibles d'évoluer dans le temps.

Justification du choix de MongoDB

MongoDB a été choisi pour sa capacité à gérer des données semi-structurées et évolutives. Son modèle orienté documents permet de regrouper, au sein d'un même document, l'ensemble des informations liées à un patient et à ses admissions. Cette approche limite l'utilisation de jointures, simplifie les requêtes et facilite l'évolution du schéma, ce qui est particulièrement adapté aux données médicales.

Automatisation, transformation et validation

La migration est réalisée via un script Python automatisé. Les données du fichier CSV sont transformées en documents MongoDB orientés patient, avec une génération d'identifiants

uniques et un regroupement des admissions. Des tests d'intégrité sont exécutés avant la migration afin de garantir la qualité des données sources, puis après l'insertion dans MongoDB afin de valider la cohérence, le volume et la structure des données migrées.

Migration des données – Script :

Objectif
- Migrer des données médicales vers MongoDB
- Modèle orienté **patient**
- Intégrité des données garantie

Pipeline
- Chargement sécurisé (.env)
- Tests avant migration
- Transformation patient / admissions
- Insertion MongoDB + index
- Tests après migration

Bonnes pratiques
- Étapes séparées
- Script automatisé et ré-exécutable
- Accès applicatif limité (lecture / écriture)

```
Fichier CSV
(données médicales)
        ↓
Tests d'intégrité
• colonnes obligatoires
• types de données
• doublons
        ↓
Transformation
• regroupement par patient
• historique des admissions
        ↓
Insertion MongoDB
• documents orientés patient
• index pour les performances
        ↓
Tests post-migration
• volumétrie
• structure des documents
```

```python
import os
import pandas as pd
from dotenv import load_dotenv
from pymongo import MongoClient
from datetime import datetime

# Charger les variables du fichier .env
load_dotenv()

MONGO_URI = os.getenv("MONGO_URI")
DB_NAME = os.getenv("DB_NAME")
COLLECTION_NAME = os.getenv("COLLECTION_NAME")
DATA_FILE = os.getenv("DATA_FILE")

# ========================
# UTILITAIRES
# ========================

def create_patient_id(name, birth_year=None):
    """Créer un identifiant unique à partir du nom et de l'année."""
    clean_name = name.replace(" ", "_").lower()
    return f"patient_{clean_name}_{birth_year if birth_year else 'unknown'}"

# ========================
# CHARGEMENT CSV
# ========================

def load_csv(file_path):
    """Charge le fichier CSV."""
    print(f"Chargement du fichier CSV : {file_path}")
    return pd.read_csv(file_path)

# ========================
# TESTS AVANT MIGRATION
# ========================

def test_csv_integrity(df):
    """Tests d'intégrité automatiques sur le CSV (avant migration)."""

    print("Tests d'intégrité AVANT migration (CSV)...")

    required_columns = [
        "Name", "Age", "Gender", "Blood Type",
        "Medical Condition", "Date of Admission",
```

Migration des données – Données dans MongoDB

Structure des données

- Données stockées sous forme de documents
- Organisation en collections MongoDB

Validation de la migration

- Contrôle du nombre de documents
- Vérification de la cohérence des champs

Résultat

- Données correctement importées
- Structure flexible et exploitable

ADD DATA | EXPORT DATA | UPDATE | DELETE

```
_id: "patient_bobby_jackson_1995"
name : "Bobby JacksOn"
age : 30
gender : "Male"
blood_type : "B-"
admissions : Array (1)
  0: Object
    medical_condition : "Cancer"
    date_of_admission : "2024-01-31"
    discharge_date : "2024-02-02"
    doctor : "Matthew Smith"
    hospital : "Sons and Miller"
    room_number : 328
    admission_type : "Urgent"
    medication : "Paracetamol"
    test_results : "Normal"
    billing_amount : 18856.281305978155
    insurance_provider : "Blue Cross"
```

```
_id: "patient_leslie_terry_1963"
name : "LesLie TErRy"
age : 62
gender : "Male"
blood_type : "A+"
admissions : Array (1)
```

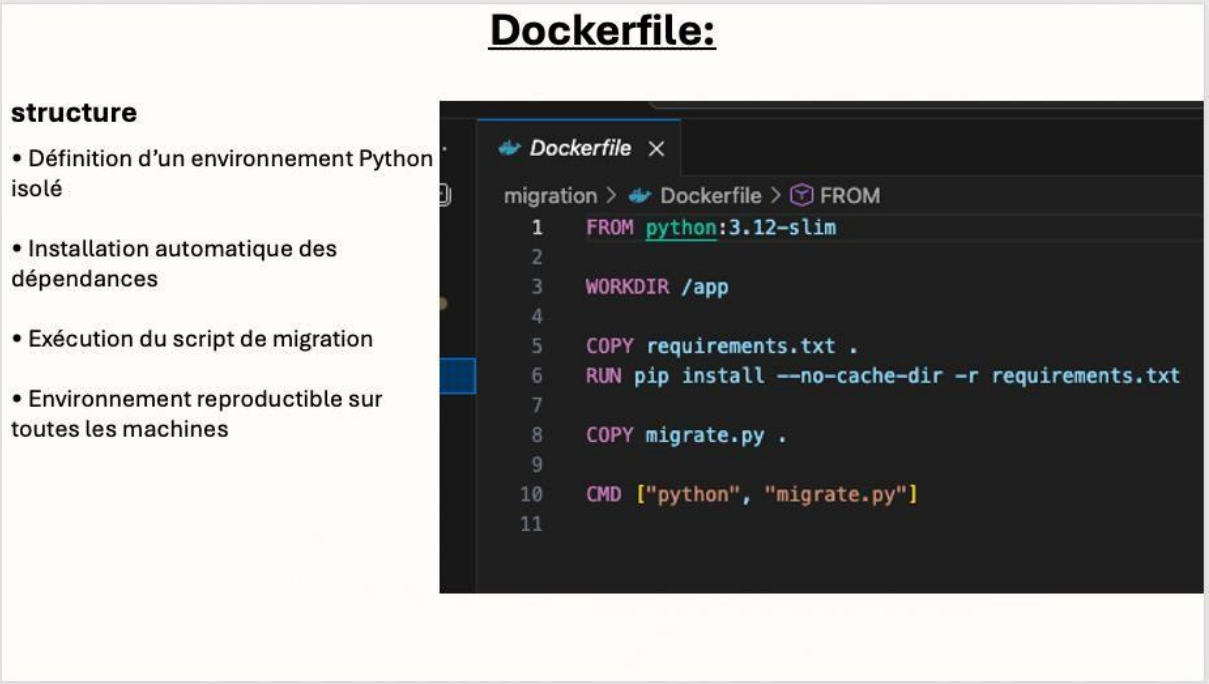
III – Conteneurisation de l'application avec Docker

Objectifs de la conteneurisation

La conteneurisation a pour objectif de garantir la portabilité, la reproductibilité et l'automatisation du processus de migration. L'utilisation de Docker permet d'exécuter la migration des données dans un environnement isolé et contrôlé, indépendamment du système hôte.

Image Docker du service de migration

Un Dockerfile dédié est utilisé pour construire une image légère basée sur `python:3.12slim`. Cette image inclut l'installation des dépendances Python nécessaires à la migration ainsi que le script de migration. Le point d'entrée du conteneur déclenche automatiquement l'exécution du script Python lors du démarrage.



Orchestration avec Docker Compose

Docker Compose est utilisé pour orchestrer les différents services de l'application. Deux services principaux sont définis :

- un service MongoDB chargé de l'hébergement de la base de données, • un service *migrator* responsable de l'exécution du script de migration.

Le mécanisme `depends_on` garantit que le service de migration s'exécute une fois MongoDB démarré.

Architecture Docker Compose

Services orchestrés

MongoDB

Script de migration

Rôle de Docker Compose

Lancement coordonné des conteneurs

Gestion des dépendances

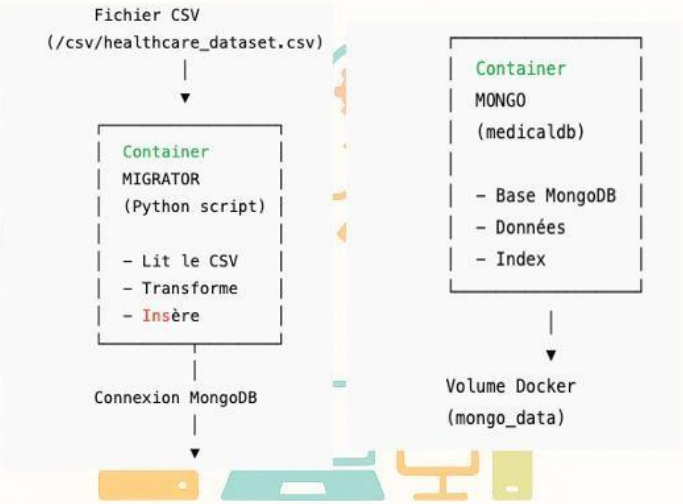
Communication entre services

Bénéfices

Architecture claire

Exécution automatisée

Base prête pour le cloud



docker-compose.yml

```yaml
version: "3.9"

services:
  mongo:
    image: mongo:7
    container_name: mongo-medical
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: root
      MONGO_INITDB_ROOT_PASSWORD: rootpassword
      MONGO_INITDB_DATABASE: medicaldb
    volumes:
      - mongo_data:/data/db
      - ./mongo/init-mongo.js:/docker-entrypoint-initdb.d/init-mongo.js:ro
    ports:
      - "27017:27017"

  migrator:
    build: ./migration
    container_name: migrator-medical
    depends_on:
      - mongo
    environment:
      MONGO_URI: mongodb://app_user:app_password@mongo:27017/medicaldb?authSource=medicaldb
      DATASET_PATH: /csv/healthcare_dataset.csv
    volumes:
      - ./data:/csv
    command: ["python", "migrate.py"]

volumes:
  mongo_data:
```

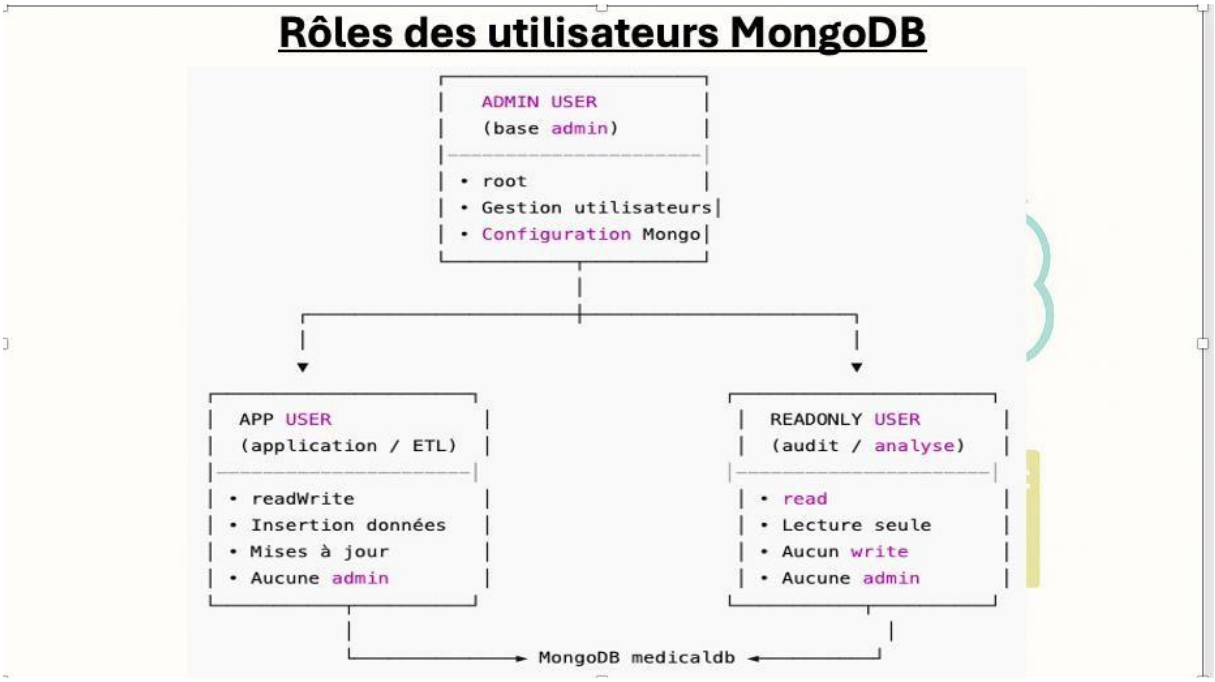
Automatisation et stockage persistant

Le pipeline de migration est entièrement automatisé lors du lancement de Docker Compose. Un volume Docker est configuré pour assurer la persistance des données MongoDB, permettant de conserver les données même après l'arrêt ou le redémarrage des conteneurs.

IV – Sécurité, authentification et bonnes pratiques

Gestion des utilisateurs et des rôles MongoDB

La sécurité de la base de données repose sur la création de comptes MongoDB distincts, chacun disposant de permissions spécifiques. Trois rôles ont été mis en place : un utilisateur administrateur disposant de droits étendus, un utilisateur applicatif dédié à la migration avec des droits en lecture et écriture sur la base médicale, et un utilisateur en lecture seule destiné aux accès de consultation.



Séparation des privilèges et contrôle des accès

Le principe du moindre privilège est appliqué afin de limiter les risques liés à un accès excessif aux données sensibles. Chaque service utilise uniquement les droits nécessaires à son fonctionnement, ce qui permet de réduire l'exposition en cas de mauvaise configuration ou d'incident.

init-mongo.js

mongo > init-mongo.js > ...

```javascript
/*
=====================
1. ADMIN (base admin)
=====================
*/
db = db.getSiblingDB("admin");

db.createUser({
  user: "admin_user",
  pwd: "admin_password",
  roles: [
    { role: "root", db: "admin" }
  ]
});

/*
=====================
2. APPLICATION / MIGRATION
=====================
*/
db = db.getSiblingDB("medicaldb");

db.createUser({
  user: "app_user",
  pwd: "app_password",
  roles: [
    { role: "readWrite", db: "medicaldb" }
  ]
});

/*
=====================
3. READ ONLY
=====================
*/
db.createUser({
  user: "readonly_user",
  pwd: "readonly_password",
  roles: [
    { role: "read", db: "medicaldb" }
  ]
});
```

Validation de l'intégrité des données après migration

Des tests automatiques sont exécutés après la migration afin de vérifier l'intégrité des données stockées dans MongoDB. Ces contrôles portent notamment sur le nombre de documents insérés, la présence des champs essentiels et la cohérence globale de la structure des documents, garantissant ainsi la fiabilité des données migrées.

migration > crud_demo.py > mongo_connection

```python
import os
from dotenv import load_dotenv
from pymongo import MongoClient

# Charger les variables d'environnement
load_dotenv()

MONGO_URI = os.getenv("MONGO_URI")
DB_NAME = os.getenv("DB_NAME")
COLLECTION_NAME = os.getenv("COLLECTION_NAME")

def mongo_connection():
    client = MongoClient(MONGO_URI)
    db = client[DB_NAME]
    return db[COLLECTION_NAME]

def create_patient(collection):
    print("CREATE : ajout d'un patient")
    patient = {
        "_id": "patient_demo_001",
        "name": "John Doe",
        "age": 45,
        "gender": "Male",
        "admissions": []
    }
    collection.insert_one(patient)

def read_patient(collection):
    print("READ : lecture du patient")
    patient = collection.find_one({"_id": "patient_demo_001"})
    print(patient)

def update_patient(collection):
    print("UPDATE : modification du patient")
    collection.update_one(
        {"_id": "patient_demo_001"},
        {"$set": {"age": 46}}
    )

def delete_patient(collection):
    print("DELETE : suppression du patient")
```

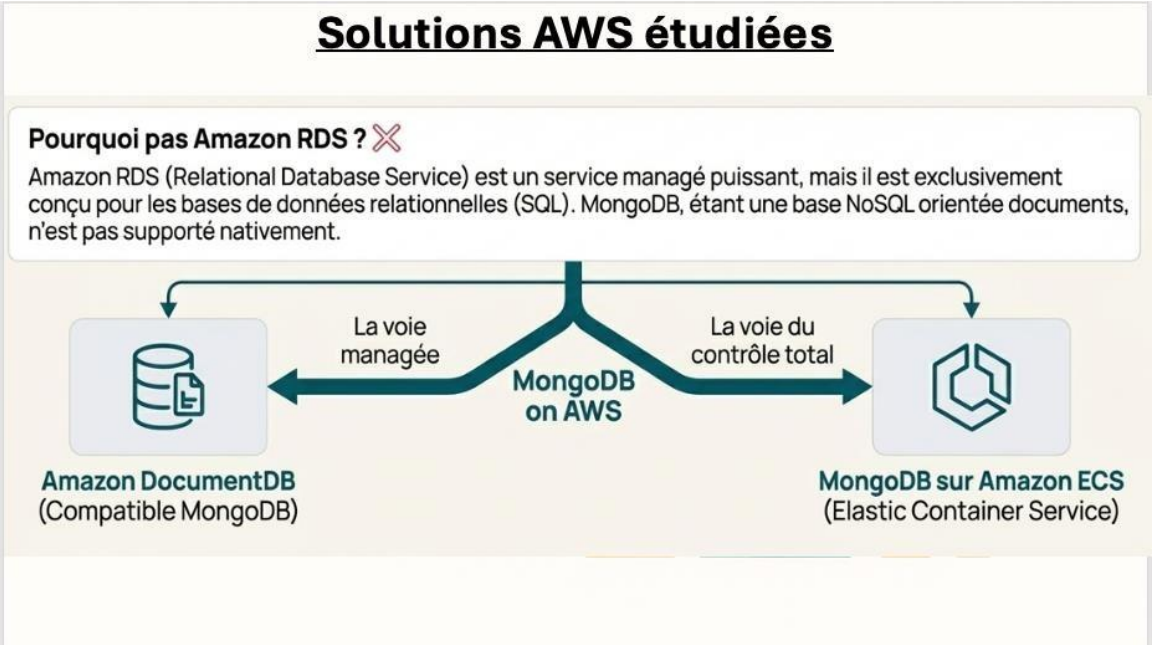
V – Étude des solutions de déploiement sur AWS

Limites d'une solution locale

Une infrastructure locale présente des limites importantes face à l'augmentation du volume de données médicales. Elle implique une gestion manuelle des ressources, une maintenance lourde, ainsi qu'un risque accru en cas de panne ou de perte de données. De plus, la scalabilité reste limitée, rendant difficile l'adaptation aux besoins futurs du client.

Services AWS adaptés à MongoDB

Plusieurs solutions AWS ont été étudiées pour héberger MongoDB dans le cloud. Amazon RDS a été exclu car il est réservé aux bases de données relationnelles et ne supporte pas MongoDB. Deux solutions principales ont été retenues : Amazon DocumentDB, solution managée compatible avec MongoDB, offrant une haute disponibilité, des sauvegardes automatiques et une maintenance simplifiée ; et le déploiement de MongoDB sur Amazon ECS via des conteneurs Docker, permettant un contrôle total sur la configuration et les versions utilisées.



Comparaison des options de déploiement cloud

Le choix entre une solution managée et une solution conteneurisée dépend des besoins du client. Amazon DocumentDB privilégie la simplicité, la fiabilité et la réduction de la charge opérationnelle, tandis que MongoDB sur ECS offre davantage de flexibilité et de maîtrise de l'infrastructure, au prix d'une gestion plus complexe. Cette analyse permet de proposer une solution cloud adaptée aux contraintes techniques, budgétaires et opérationnelles du client.

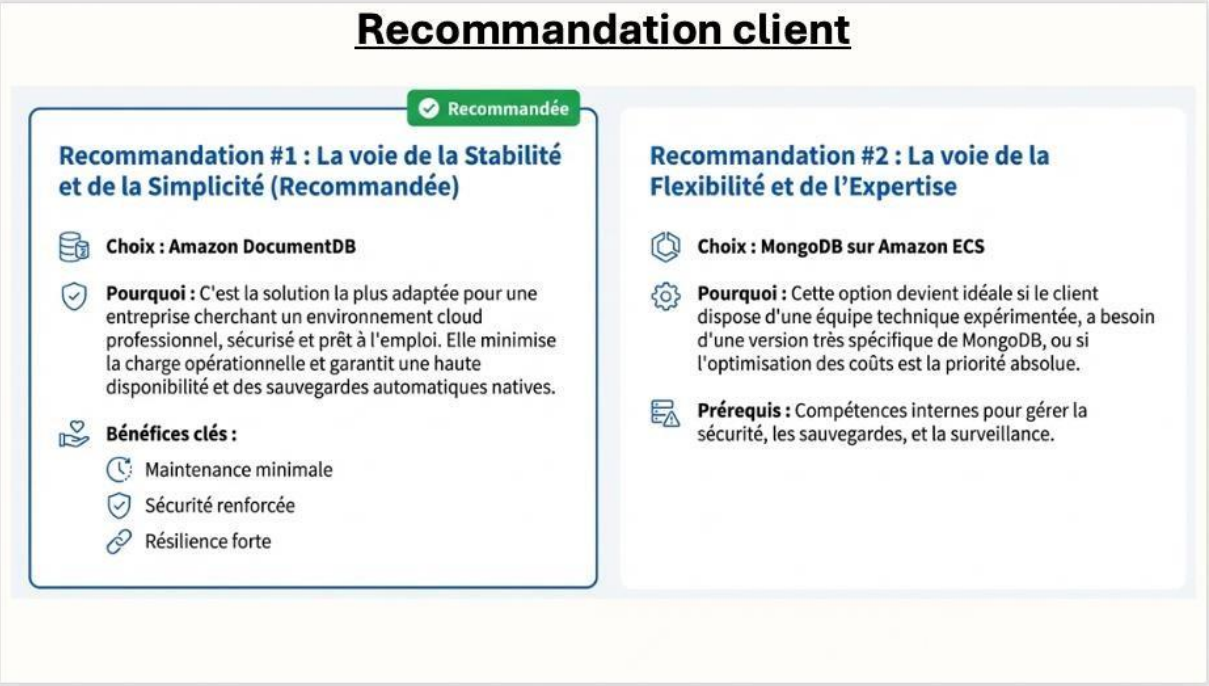
Solutions AWS étudiées

Critère	Amazon DocumentDB (Managé)	MongoDB sur ECS (Contrôlé)
Effort de Gestion	**Minimal** (Mises à jour, patchs, sauvegardes gérés par AWS)	**Élevé** (Responsabilité interne pour la maintenance, les backups et la sécurité)
Contrôle & Flexibilité	**Limité** (Version et configuration gérées par AWS)	**Total** (Choix de la version, plugins, configuration fine)
Haute Disponibilité	**Intégrée** (Architecture Multi-AZ native)	**À configurer** (Nécessite une architecture manuelle avec réplicas)
Coût	**Prévisible mais plus élevé** (Paiement pour le service managé)	**Potentiellement plus bas**, mais variable (Dépend de l'optimisation)
Scalabilité	**Automatisée** (Stockage et réplicas en lecture)	**Manuelle** ou **à scripter** (Nécessite de configurer l'Auto Scaling)
Idéal pour...	Équipes visant la **stabilité** et la **rapidité** de mise en production.	Équipes expertes cherchant un **contrôle total** et une **optimisation des coûts**.

VI – Recommandation d'architecture et services essentiels

Recommandation de déploiement

À l'issue de l'étude des solutions AWS, **Amazon DocumentDB est recommandé pour le client**. Cette solution managée répond aux besoins de stabilité, de sécurité et de simplicité opérationnelle. Elle permet de limiter la charge de maintenance tout en garantissant une haute disponibilité et des sauvegardes automatiques, ce qui en fait un choix particulièrement adapté à un environnement de données médicales.



Services essentiels garantis par AWS

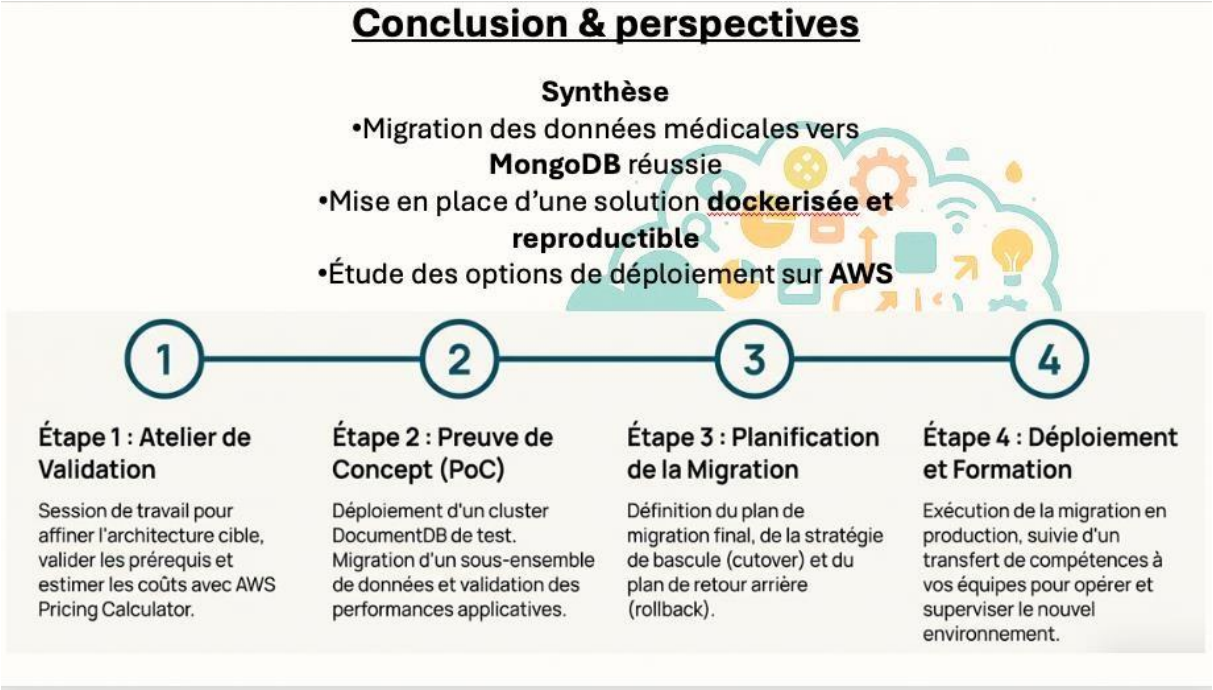
Indépendamment de la solution retenue par le client, AWS fournit un socle commun de services garantissant un environnement de production fiable. Cela inclut l'isolation réseau via des VPC, des contrôles d'accès sécurisés avec IAM appliquant le principe du moindre privilège, ainsi que le chiffrement des données au repos et en transit.

AWS met également à disposition des mécanismes de sauvegarde intégrés et des outils de surveillance centralisés, notamment Amazon CloudWatch, permettant le suivi en temps réel des performances et la détection proactive des incidents. Ces services constituent un socle essentiel de sécurité, de fiabilité et de supervision, quelle que soit l'architecture de déploiement choisie.

VII – Conclusion et perspectives

Conclusion

Ce projet a permis de mener à bien la migration de données médicales vers MongoDB en mettant en place une solution automatisée, dockerisée et reproductible. L'étude des options de déploiement sur AWS a permis d'identifier une architecture cloud adaptée aux enjeux de scalabilité, de sécurité et de fiabilité, avec une recommandation claire en faveur d'Amazon DocumentDB.

Perspectives

Dans une logique de mise en production, plusieurs étapes peuvent être envisagées : la validation de l'architecture cible, la réalisation d'un proof of concept sur AWS, la planification détaillée de la migration finale ainsi que le déploiement en production accompagné d'une formation des équipes. Ces étapes permettront de sécuriser la transition vers le cloud et d'assurer une exploitation durable et maîtrisée de la solution.